Lab 11 — DoS Simulation and Detection

You will generate a controlled SYN-flood against your own loopback, watch the connection table fill, then mitigate with `iptables` rate-limiting.

Run on https://killercoda.com/playgrounds/scenario/ubuntu

Required software (free, apt): - `hping3` - `nginx` - `iptables` (pre-installed)

Step 1 — Install attack and target

```
apt update && apt install -y hping3 nginx
```

```
systemctl start nginx
```

```
ss -tlnp | grep :80
```

Step 2 — Baseline the open connections

```
ss -s
```

Note the `TCP:` line — typical baseline is ~10 sockets.

Step 3 — Launch a SYN flood (loopback only!)

```
timeout 10 hping3 -S -p 80 --flood --rand-source 127.0.0.1 &
sleep 3
ss -s | head
```

`SYN-RECV` count explodes. The kernel is holding half-open connections waiting for the ACK that never comes.

Step 4 — Detect with `iptables` counters

```
iptables -A INPUT -p tcp --syn --dport 80 -j LOG --log-prefix "SYN: "
--log-level 4
iptables -L INPUT -v -n | head
```

The packet counter on the rule rises by tens of thousands per second.

Step 5 — Mitigate with rate-limit + SYN cookies

```
sysctl -w net.ipv4.tcp_syncookies=1
```

```
iptables -I INPUT -p tcp --syn --dport 80 -m limit --limit 25/s --
limit-burst 50 -j ACCEPT
```

```
iptables -A INPUT -p tcp --syn --dport 80 -j DROP
```

`tcp_syncookies` makes the kernel respond statelessly during a flood. The `limit` module caps how many SYNs per second nginx will see.

Re-run the flood and watch the difference:

```
timeout 10 hping3 -S -p 80 --flood --rand-source 127.0.0.1
```

```
ss -s
```

Step 6 — Distinguish DoS from DDoS

Signal	DoS	DDoS
Source IPs in `tcpdump`	One	Thousands
Mitigation possible at host	Yes (rate-limit)	No (need upstream)
Typical defence	iptables / fail2ban	Cloud scrubber

Free online tools

- **Cloudflare radar** — live DDoS observability: https://radar.cloudflare.com

- **DDoS-Guard public threat map** — https://ddosguard.net
- **Project Shield** (free for journalists/NGOs) — https://projectshield.withgoogle.com
- **MITRE ATT&CK Network DoS technique** — https://attack.mitre.org/techniques/T1498/

What you learned

- Why SYN floods exhaust resources without sending real data.
- That two kernel settings (`tcp_syncookies` + rate-limit) blunt host-level floods.
- Why volumetric DDoS must be stopped *upstream* of your link.